Методы и программные продукты для повышения производительности проектов на базе ПЛИС Xilinx

Илья ТАРАСОВ,
к. т. н.
tile@kc.ru

Постоянный рост логического объема и тактовой частоты микросхем программируемой логики имеет и оборотную сторону. При разработке сложных и комплексных проектов на базе ПЛИС большого объема получаемые характеристики могут оказаться существенно хуже, чем заявляемые производителем как максимальные. Это может происходить как по объективным причинам, так и из-за некорректно выполненного описания проекта, которое и привело к получению неоптимального решения. В данной публикации рассмотрен ряд методик, способствующих достижению максимальной производительности проектов на базе ПЛИС Xilinx.

Введение

Говоря о производительности ПЛИС, необходимо четко различать пиковые характеристики кристалла и производительность в конкретной задаче, с учетом реализованной топологии программируемых соединений. Для сравнения рассмотрим понятие «производительность» применительно к микропроцессорным системам. Для ее оценки можно пользоваться целым рядом количественных характеристик, таких как тактовая частота процессора и памяти, объем кэша и его характеристики, временные задержки при блочном и произвольном доступе в память, тип внешних интерфейсов и пр. Тем не менее для сложной процессорной системы обычно не удается абсолютно точно спрогнозировать время выполнения того или иного фрагмента кода, поскольку на него оказывают влияние очень много факторов. Например, одна и та же программа, запущенная в многозадачной ОС, может как целиком попасть в кэш процессора, так и оказаться частично выгруженной на диск, если одновременно с ней выполняется работа большого количества других программ и системных служб. Поэтому при неудовлетворительной производительности программы пользователь PC обычно задумывается о замене процессора на более мощный, с большим объемом кэша, а также об увеличении объема оперативной памяти.

В то же время понятие «максимальная тактовая частота» соответствует в ПЛИС ситуации, когда все программируемые соединения выполнены оптимальным образом. Следовательно, один раз выполненный проект на базе ПЛИС будет сохранять свои характеристики производительности в течение всего времени функционирования. Можно говорить только о более или менее удачно выполненной трассировке соединений, которая и определит максимально возможную для данного проекта частоту. В зависимости от стиля описания, выбранных компонентов, настроек САПР и других факторов получаемая тактовая частота может варьироваться в довольно широких пределах. Эти пределы вполне могут перекрывать градации скорости ПЛИС, и, таким образом, в ряде случаев появляется возможность для более медленного (и дешевого) кристалла путем применения корректных методик описания и оптимизации проекта добиться тех же результатов, что и для более быстрого и дорогого, к которому такие методики не применялись. Поэтому очень важно владеть инструментами и методами улучшения характеристик проекта на базе ПЛИС.

Порядок улучшения характеристик проекта показан на рис. 1. Эти рекомендации составлены на основе учебных материалов фирмы Xilinx и в полном виде приводятся в официальных курсах повышения квалификации в рамках программы авторизованных тренинг-партнеров.

Далее рассмотрим отдельные этапы показанной схемы.

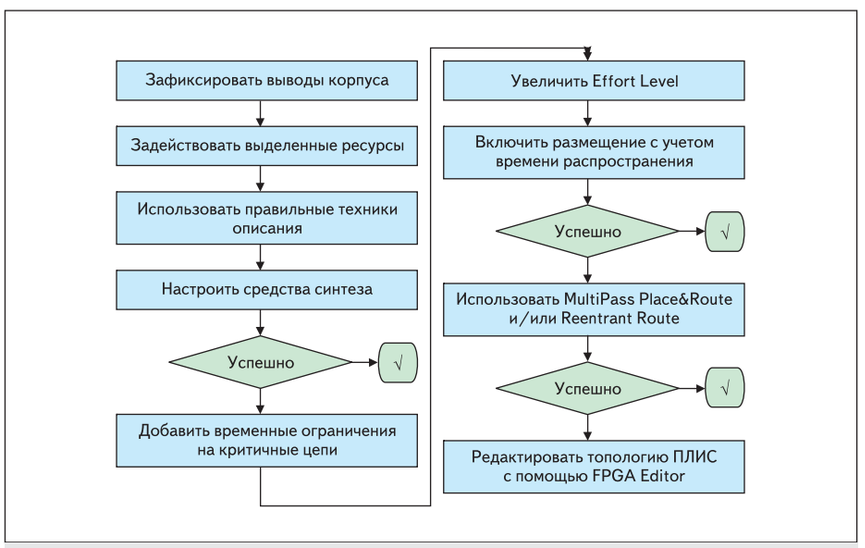

Рис. 1. Порядок улучшения характеристик проекта на базе ПЛИС Xilinx

Фиксация выводов корпуса

Несмотря на то что программируемые пользователем выводы взаимозаменяемы, для высокопроизводительных проектов может оказаться важным обеспечение правильной компоновки внешних сигналов и шин. Очевидно, что сигналы должны подаваться на выводы ПЛИС возможно ближе к месту их реального использования на кристалле. Однако существуют также рекомендации, связанные с особенностями конструкции логической ячейки и распределения трассировочных линий. Если смотреть на топологию ПЛИС в том виде, в котором она представлена в инструментах САПР ISE, то внутри логических ячеек данные распространяются слева направо, а снизу вверх проложены цепи ускоренного переноса, предназначенные для построения арифметических модулей и счетчиков (где и требуется передача бита переноса

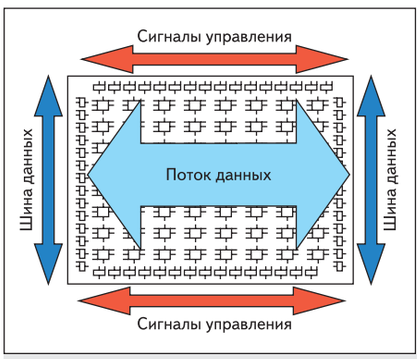

Рис. 2. Рекомендуемое расположение внешних выводов проекта для ПЛИС малого и среднего объема

в следующий разряд). Поэтому целесообразно размещать ПЛИС на печатной плате таким образом, чтобы шина данных обрабатываемого сигнала входила слева и проходила «сквозь» матрицу логических ячеек, выходя с правой стороны. При этом верхнюю и нижнюю стороны оказывается возможным отвести для сигналов управления (рис. 2). Внутри шины данных рекомендуется располагать младшие биты внизу, а старшие, соответственно, вверху корпуса ПЛИС, поскольку это соответствует естественному направлению внутренних линий ускоренного переноса.

Для ПЛИС большого объема существуют некоторые уточнения к порядку рекомендуемого назначения выводов. Например, чтобы сложить младшие разряды двух шин, необходимо подать их на вход какой-то одной логической ячейки. Для ПЛИС небольшого объема, которые, соответственно, выпускаются в небольших корпусах, эта проблема стоит не слишком остро, но большой объем (и большой корпус) могут привести к тому, что данные, обрабатываемые одной логической ячейкой, отстоят друг от друга слишком далеко вдоль одной из сторон ПЛИС. В этом

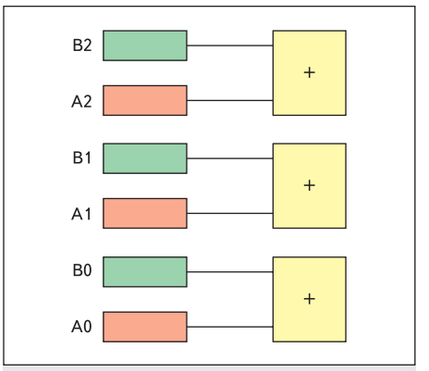

Рис. 3. Рекомендуемое расположение внешних выводов проекта для ПЛИС большого объема

случае рекомендуется подавать такие сигналы с чередованием шин, как показано на рис. 3. В этом случае при необходимости сложения (а также вычитания или любого иного способа совместной побитной обработки шин) трассировочные линии от выводов ПЛИС до соответствующих логических ячеек будут проложены по максимально короткому пути.

Такая ранняя фиксация выводов корпуса является следствием тщательно проведенного планирования структуры проекта и взаимного расположения компонентов на печатной плате. Как правило, даже для сложных проектов, содержащих большое количество внешних интерфейсов и микросхем, в том числе и скоростных, можно найти приемлемое решение с точки зрения топологии печатных проводников, их кратчайшей длины, удобного размещения и общей помехоустойчивости. Для дифференциальных сигналов необходимо сразу учитывать, что объединение выводов ПЛИС в пары возможно для строго определенных пар. Конкретнее — каждый из программируемых выводов имеет свой «парный» дифференциальный вывод, причем полярность внутри этой пары строго определена.

Ряд выводов ПЛИС могут выступать в качестве входов тактового сигнала, причем внутри эти входы подаются на специальные блоки формирования тактовой частоты, использование которых настоятельнейшим образом рекомендуется в любых проектах. Если же речь идет о высокоскоростных разработках, использование специализированных входов для подачи тактового сигнала является обязательным. Такие входы помечены в документации как GCLK (Global Clock) и имеются в ПЛИС в количестве, равном количеству блоков формирования тактовой частоты (2, 4, 8, 16). Входы равнозначны в смысле быстродействия, так что GCLK0 не имеет преимуществ перед GCLK1.

Фиксация выводов корпуса может быть удобно проведена с помощью программы PACE (Pins & Area Constraints Editor). Внешний вид окна этой программы показан на рис. 4.

Проектные ограничения сохраняются в файле с расширением ucf. Информация о назначенных выводах корпуса хранится в текстовом виде и может быть легко отредактирована вручную или создана автоматически при помощи других инструментов — например, если разработчик использует собственные программы, автоматически отслеживающие изменения в печатной плате. Визуализация расположения выводов ПЛИС относительно матрицы логических ячеек помогает разработчикам назначить выводы не только оптимально относительно матрицы логических ячеек, но и удобно для их дальнейшей трассировки на печатной плате.

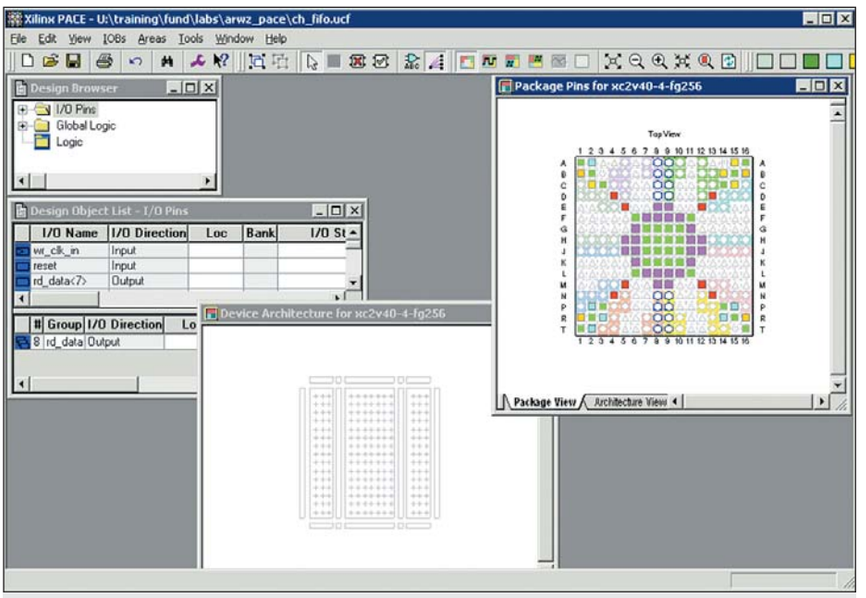

Рис. 4. Внешний вид программы PACE

Использование встроенных аппаратных ресурсов ПЛИС

Ряд цифровых узлов в составе ПЛИС обладают фиксированно высокой производительностью «благодаря конструкции». Речь идет о выделенных аппаратных ресурсах, которые введены на кристалл ПЛИС для решения часто встречающихся задач. Поскольку эти узлы выполнены с помощью обычных металлических соединений, к ним неприменимы рассуждения о потере производительности ввиду задержек на коммутационных элементах или неудачной трассировки. К таким «производительным по определению» устройствам относятся:

- блочная память;
- аппаратные умножители;
- тактовые линии;
- процессорные ядра PowerPC;
- приемопередатчики MultiGigabit Transceivers (Ethernet MAC, SATA, PCI Express);
- сериализаторы/десериализаторы блоков ввода/вывода.

Блочная память

Блочная память представляет собой двупортовую синхронную память с организацией 1024×18 бит в семействах Spartan-3 (всех разновидностей) и Virtex-4. В семействе Virtex-5 применены укрупненные блоки с организацией 1024×36 бит. Блоки оснащены дополнительными схемами, помогающими организовать FIFO с аппаратным контролем исчерпания и переполнения.

Аппаратные умножители

Аппаратные умножители являются весьма эффективным средством реализации на базе ПЛИС высокопроизводительных вычислительных устройств. Начиная с семейств Virtex-II и Spartan-3, FPGA получили скоростные аппаратные умножители 18-разрядных операндов. Тот факт, что даже устройства начального уровня содержали 4 умножителя, способных работать на частоте до 300 МГц, позволил реализовать на ПЛИС достаточно мощные устройства цифровой обработки сигналов. Последние семейства содержат до 640 (Virtex-5SXT) умножителей 18×25 = 48 (т. е. перемножаются 18- и 25-разрядные операнды, результат помещается в 48-разрядный аккумулятор). Совмещение умножителя и аккумулятора позволяет получать гарантированно высокие частоты обработки сигналов в достаточно большом классе алгоритмов. Для Virtex-5 максимальная тактовая частота достигает 550 МГц, что в сочетании с большим количеством блоков «умножение с накоплением» дает показатель 352 млрд умножений с накоплением в секунду! Важно, что тактовая частота самого блока не зависит от особенностей трассировки конкретного проекта.

Тактовые линии

Тактовые линии представляют собой проводники, проложенные по кристаллу ПЛИС таким образом, чтобы доставлять тактовый сигнал ко всем синхронным ресурсам кристалла (триггеры, блочная память, умножители и т. д.) с минимальной задержкой распространения. Эти сигналы формируются специальными блоками (DCM — Digital Clock Manager, CMT — Clock Management Tile), которые располагаются вблизи выделенных выводов корпуса. В этих блоках широко используются схемы автоподстройки задержки и фазы (Virtex-5 обеспечивает как DLL, так и PLL, объединяемые в новом блоке Clock Management Tile), что помогает получить тактовый сигнал, одновременно приходящий всем областям ПЛИС. Все это гарантирует отсутствие «перекосов тактового сигнала» и «гонок фронтов», что встречается при формировании тактового сигнала с помощью логических ячеек или входов общего назначения.

Процессорные ядра PowerPC-405

Процессорные ядра PowerPC-405 введены на кристалле в виде hard-IP — фрагментов топологии, лицензия на которые приобретена у IBM. Таким образом, производительность этих ядер достаточно высока и определяется характеристиками аппаратного ядра, а не матрицы логических ячеек. Процессоры PowerPC имеются в отдельных семействах — Virtex-II Pro, Virtex-4 FX, Virtex-5 FXT. Использование процессора в качестве основы проекта на базе ПЛИС позволяет в полной мере использовать возможности современных процессорных комплексов. Поддержка gcc позволяет, например, реализовать проект под управлением ОС Linux (всего же количество ОС для PowerPC исчисляется десятками). В сочетании с возможностями матрицы логических ячеек производительность такой системы может быть существенно увеличена.

Приемопередатчики MGT

Приемопередатчики MGT (Multi-Gigabit Transceivers) также являются аппаратными блоками, реализующими широко распространенные протоколы последовательного обмена данными. На кристаллах (Virtex-4 FX, Vurtex-5 LXT, Virtex-5 SXT, Virtex-5 FXT) расположено от 2 до 6 таких модулей, которые могут функционировать в качестве Gigabit Ethernet, а в Virtex-5 еще и в качестве PCI-Express или SATA. Размещение на кристалле аппаратного модуля такого типа несущественно увеличивает его стоимость, однако позволяет сэкономить логические ячейки. Например, для реализации контроллера Ethernet с помощью программируемых ячеек их потребуется около 5 тыс. Наличие эффективных аппаратных средств передачи больших объемов данных упрощает организацию системы в целом. Важно и то, что разработчик избавлен от необходимости отслеживать нюансы таких протоколов и реализовать их самостоятельно.

Сериализаторы/десериализаторы

Сериализаторы/десериализаторы встроены в блоки ввода/вывода и позволяют организовывать высокоскоростную передачу данных через обычные программируемые линии ввода/вывода. В качестве постановки проблемы можно обратиться к цифре 1,25 Гбит/с, которую допускают устройства Virtex-5. Очевидно, что матрица логических ячеек не сможет обеспечить поток данных, работая с частотой 1,25 ГГц (системная тактовая частота составляет всего 550 МГц). Однако сериализатор, встроенный в каждый блок ввода/вывода, позволяет принимать данные побайтно (вообще, разрядность регулируется в пределах 6–10), аппаратно формируя последовательную посылку. Таким образом, программируемая часть ПЛИС может подавать 8-разрядные данные на такой сериализатор с частотой около 150 МГц, что является вполне достижимым показателем. Аналогично функционирует и десериализатор, преобразующий высокоскоростной входной последовательный поток в параллельные данные для последующего чтения их на приемлемой частоте.

Таким образом, при предварительном планировании структуры проекта, выборе ПЛИС и оценке производительности, можно в первую очередь отталкиваться от характеристик аппаратно реализованных модулей, которые обеспечивают гарантированно высокую производительность и хорошее соотношение производительность/цена.

Что касается программируемых ячеек, то существуют общие рекомендации по удержанию производительности проекта на максимально достижимом уровне. Главное требование, отнюдь не всегда достижимое на практике, заключается в сохранении только одного логического уровня в комбинаторных выражениях. Иными словами, при передаче сигнала от одного триггера к другому на этом пути должно быть не более одной логической ячейки. В этом случае тактовая частота проекта может равняться значению, приводимому в качестве «системной тактовой частоты» (например, для Virtex-5 это 550 МГц). Кроме того, при построении счетчиков следует ограничивать их разрядность — устройства серии Virtex способны поддерживать максимально достижимую тактовую частоту только для счетчиков с 14 и менее разрядами. Это требование появляется потому, что линии ускоренного переноса хотя и являются достаточно быстрыми, имеют все же вполне определенную задержку распространения сигнала.

ПЛИС Virtex-5 неочевидным образом помогают построению скоростных схем на логических ячейках. Дело в том, что их логические генераторы являются 6-, а не 4-входовыми, и следовательно, они способны реализовать более сложные логические выражения с помощью одного генератора. Значит, в каких-то случаях предыдущие поколения FPGA будут вынуждены использовать

еще одну логическую ячейку (и тогда уже нельзя говорить о достижении теоретического максимума тактовой частоты), а Virtex-5 все еще будет способен реализовать такую функцию в одной ячейке.

Проектирование синхронных устройств

Постоянное повышение тактовой частоты цифровых микросхем имеет и свою оборотную сторону — период сигнала становится соизмеримым с временем переходных процессов и задержками распространения сигналов по кристаллу. Поэтому при проектировании и анализе цифровых схем необходимо учитывать множество различных эффектов, которыми ранее можно было пренебречь. Причем это касается не только проектов, изначально рассчитанных на высокую тактовую частоту, но и обычных изделий, выполненных на базе 90- или 65-нм микросхем (таких, как Spartan-3, Virtex-4, 5).

На рис. 5 показана иллюстрация к серьезной проблеме в высокочастотных цифровых схемах — метастабильности. Метастабильностью называется «промежуточное» состояние триггера, в которое он может попасть при несоблюдении требуемых временных интервалов на его входах. В этом состоянии выход триггера может находиться в «промежуточном» уровне, который через некоторое время все же придет к одному из устойчивых логических состояний, но это время строго не определено и может быть существенно больше периода тактового сигнала.

Метастабильность носит статистический характер, и для снижения вероятности ее возникновения следует избавиться от переключений данных непосредственно перед фронтом тактового сигнала (т. е. во время t_{setup} логический уровень на входе D не должен меняться). В то же время в асинхронных схемах, где не существует общего отсчета времени для всех узлов, моменты смены сигналов могут существенно зависеть от особенностей конкретного варианта трассировки, и в том числе происходить непосредственно перед фронтом, по которому происходит запись в триггер. Нестабильность тактовой частоты, jitter («дрожание» положений перепадов тактового сигнала во времени), переходные процессы, температурная нестабильность параметров кристалла — все это факторы, которые резко увеличивают вероятность переключения данных в неподходящие моменты.

Для снижения вероятности появления метастабильности необходимо обеспечить устойчивый логический уровень на входе данных с его гарантированной сменой вне пределов setup и hold. Такое поведение схемы может быть получено при синхронном проектировании, когда все сигналы внутри микросхемы изменяются строго по перепаду тактовой частоты. В сочетании с конечным временем распространения сигнала можно

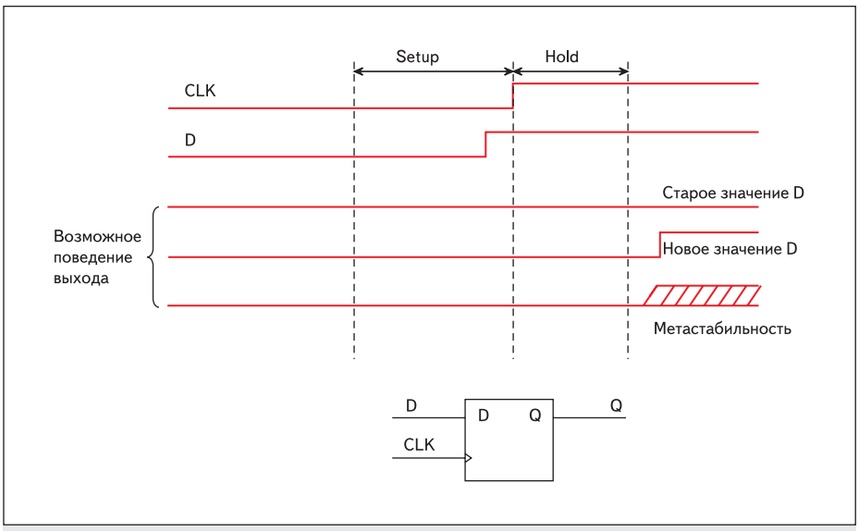

Рис. 5. Иллюстрация к проблеме метастабильности

практически гарантировать соблюдение t_{hold}, а что касается t_{setup}, то для него достаточно не превышать максимально допустимого значения тактовой частоты. Важно, что САПР ПЛИС с высокой точностью моделируют эти ситуации, учитывая и характеристики логических ячеек, и топологию трассировки, и такие факторы, как время нарастания тактового сигнала, jitter, нестабильность тактовой частоты, влияние температуры и напряжения питания. Поэтому предпочтительнее ориентироваться именно на полностью синхронные проекты. Это означает, что:

- все изменения данных происходят по одному перепаду тактового сигнала (обычно по фронту);
- в проекте имеется только один тактовый сигнал (для больших проектов возможно существование нескольких тактовых подсетей);
- используется только синхронный сброс;
- отсутствуют тактовые сигналы, сформированные комбинаторной логикой;
- все асинхронные входные сигналы синхронизированы.

Последний пункт нуждается в некотором пояснении. Под синхронизацией в данном случае понимается привязка момента переключения входного сигнала к тактовой частоте во избежание метастабильного поведения основных триггеров кристалла. На рис. 6 показаны основные схемы синхронизации. Даже при возникновении метастабильности логический уровень на выходе такой схемы будет считываться устойчиво. Первый из вариантов является наиболее простым, однако он не способен захватывать сигналы, длительность которых меньше периода тактовой частоты (за исключением случаев, когда этот сигнал появляется непосредственно перед фронтом clk, однако такое нестабильное поведение не может считаться приемлемым). Вторая схема при помощи логического вентиля, реализующего обратную связь, «удостоверяется», что на выходе третьего триггера действительно установился высокий логический уровень, и только после этого сбрасывает входной триггер при помощи входа асинхронного сброса. В обоих случаях серым цветом выделен триггер, который собственно и выполняет защиту от метастабильности. Сигнал на его входе оказывается привязанным к тактовому сигналу, поэтому вероятность возникновения переходных процессов непосредственно перед тактовым фронтом достаточно мала.

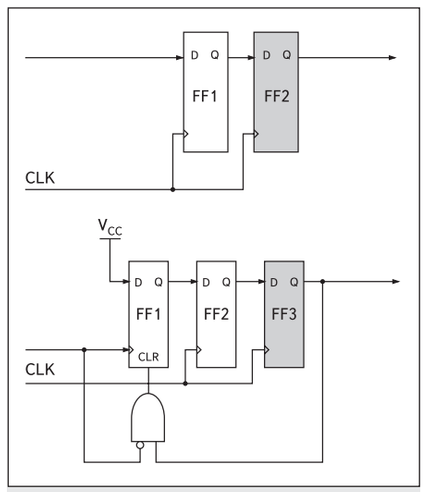

Рис. 6. Варианты схем синхронизации

При выполнении указанных выше условий достижение требуемой рабочей частоты может быть выполнено САПР и автоматически, с соблюдением всех необходимых временных интервалов. Достаточно указать период тактового сигнала (в файле пользовательских ограничений), при этом САПР сможет определить, соответствует ли выполненная трассировка требуемым условиям.

Конвейеризация

Если выполняемые над входными сигналами преобразования таковы, что реализовать их в одной логической ячейке принципиально невозможно (например, требуется слишком много входов логического генератора), в проекте неизбежно появляется многоуровневая схема. Задержка распространения сигнала через две последовательные ячейки (плюс трассировка), очевидно, больше, чем через одну, и тактовая частота проекта уменьшается в 2 и более раза. Во избежание этого можно использовать конвейерную схему. При этом вместо комплексного преобразования сигнала в одной сложной схеме используются последовательные простые операции, каждая из которых выполняется в своем цифровом узле, а промежуточные результаты запоминаются в триггерах (рис. 7). Упрощение преобразований позволяет уменьшить число последовательных ячеек от триггера до триггера, повысив, таким образом, частоту. Правда, взамен появляется задержка в появлении результата на выходе всей схемы (latency, латентность). Однако этот эффект не слишком важен в потоковых вычислениях, когда через небольшое время после старта появляется выходной поток данных, чуть запаздывающий относительно входного. Гораздо важнее в этом случае, что общая тактовая частота оказывается в разы больше, чем при попытке провести все преобразования за один такт.

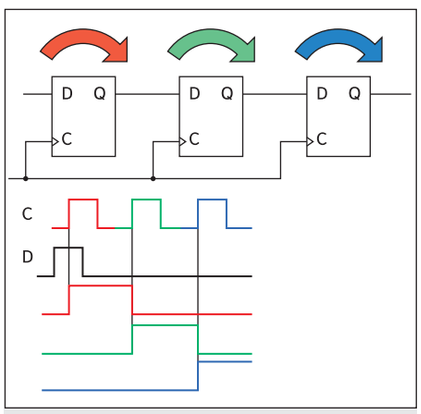

Рис. 7. Конвейеризация в синхронном проекте

Для определения возможности конвейеризации необходимо изучить отчеты САПР, обращая внимание на параметр logic levels, который и показывает количество последовательно соединенных логических ячеек. Этот параметр определяется для каждой тактовой цепи и теоретически может быть равен 1. Следует обращать внимание, какие цепи и устройства присутствуют в списке связей, поскольку для линий ускоренного переноса logic levels может быть и больше без значимой потери производительности.

Конвейеризация представляет собой весьма эффективный метод повышения производительности цифровых устройств, который широко используется в процессорах, устройствах цифровой обработки сигналов (в том числе мультимедийных), микросхемах памяти и др. Основным признаком возможности введения конвейера является наличие потоковых, непрерываемых преобразований (например, однообразных операций над отчетами, представляющими собой звук или видео). Конвейерные схемы достаточно чувствительны к необходимости смены порядка операций или отмены уже запущенных, но не выполненных до конца действий. Показательным примером конвейеризованного устройства является модуль динамической памяти (DDR SDRAM). В то время как тактовая частота, подаваемая на такой модуль, достаточно велика, динамическая память неспособна обеспечить высокую скорость случайного доступа, поскольку в этом случае требуется инициализация встроенного в микросхему контроллера, выполняющего адресацию строки и столбца ячеек памяти. Число тактов для обращения «с нуля» к произвольной ячейке может быть достаточно велико, однако последующие обращения к соседним адресам уже не требуют большого числа тактов. Поэтому динамическая память используется в сочетании с быстрой статической памятью (кэш-памятью), которая обеспечивает буферизацию произвольных обращений к объектам малого размера, а для потоковых пересылок крупных блоков данных из DDR накладные расходы на инициализацию обращения к блоку играют уже меньшую роль.

Разделение проекта на субмодули

Проекты, выполняемые в FPGA среднего и большого объема, могут получить существенный выигрыш, как в скорости трансляции, так и в производительности готового решения, при использовании иерархического разбиения на отдельные модули. Последние версии САПР ISE имеют возможность проводить трансляцию таких модулей (называемых Partitions) по отдельности. Введенная в ISE 9.x методика SmartCompile позволяет существенно сократить время трансляции, если изменения коснулись не всех модулей. Кроме того, при трансляции большого проекта часто оказывается, что отдельные его части требуют разных подходов к получению оптимального решения. Например, в отдельных случаях целью оптимизации может быть получение максимальной частоты, а в других (например, вспомогательные узлы, работающие на более низкой частоте) целесообразнее использовать оптимизацию по занимаемым ресурсам кристалла. Поэтому раздельная трансляция таких модулей с их последующим объединением позволит использовать преимущества обоих подходов.

Кроме чисто технических преимуществ, повышается читаемость проекта (за счет сокращения объема описания, подлежащего изучению в каждом из модулей), появляется возможность повторного использования модулей в других проектах, облегчается верификация и тестирование.

Исходя из архитектуры логических ячеек FPGA, рекомендуется записывать результаты работы модулей в триггеры, т. е. следовать преимущественному пути распространения сигналов в логических ячейках, когда сигнал сначала проходит через логический генератор, а затем, при необходимости, запоминается в триггере. Также можно предусмотреть и входные триггеры для повышения глубины конвейеризации.

Корректное описание устройства

Несмотря на то что современные языки описания аппаратуры, такие как VHDL и Verilog, по синтаксису и способу работы с ними приближаются к языкам программирования для ЭВМ, существует множество различий, которые обусловливают необходимость четкого понимания, как именно будет реализована та или иная синтаксическая конструкция в HDL. Интересной проблемой является и то, что долгое развитие языков программирования привело в конечном итоге к появлению достаточно эффективных компиляторов, формирующих оптимизированный код. Особенно это касается платформы x86, которая в силу своей широкой распространенности постоянно является сферой приложения усилий самых разных компаний — производителей инструментального программного обеспечения. В конечном итоге сформировалось мнение, что программа на языке высокого уровня может быть эффективно преобразована в машинные коды, поскольку существующие компиляторы включают в себя эффективные алгоритмы оптимизации. Однако перенесение такого подхода на HDL на сегодняшний день далеко не всегда корректно. Разумеется, и в среде программистов существует мнение, что для критичных фрагментов кода следует использовать язык ассемблера, дающий программисту полную уверенность в способе реализации тех или иных вычислений (мнение, которое имеет множество аргументов «за» и «против»), однако в случае с HDL речь идет как раз о принципиально другой ситуации в алгоритмах синтеза.

Прежде всего, языки описания аппаратуры являются по сути языками моделирования, а не синтеза. Синтезируемые конструкции представляют собой подмножества как VHDL, так и Verilog, а для описания структуры ПЛИС они были выбраны (точнее, адаптированы) в силу наличия соответствующих методических материалов и возможности простого переноса проектов на другую аппаратную платформу (БМК, заказные микро-

схемы). Поскольку одинаковое поведение некоторого цифрового узла может быть получено разными способами (с использованием разных ресурсов ПЛИС), возникает некоторая неопределенность в способах реализации таких узлов. Ввиду того, что VHDL и Verilog изначально не были специально ориентированы на описание ПЛИС, средствам синтеза приходится принимать решение на основе либо внутренних настроек, либо базируясь на нюансах описания, встреченного в исходном тексте. В конечном итоге возникает ситуация, когда несколько синтаксически эквивалентных фрагментов текста будут формировать одинаковые по поведению, но различные по производительности цифровые узлы. При этом сама по себе возможность реализовать схему различными способами не позволяет выбирать оптимальные варианты, как это возможно для компиляторов языков высокого уровня. Проблема усугубляется также и тем, что развитие и смена архитектур FPGA, появление дополнительных устройств (и вообще сам факт возможности внесения на кристалл более производительных и удобных в работе узлов и систем без потери совместимости с HDL) снижает ценность «универсальных средств синтеза». Гораздо важнее оказывается своевременная замена САПР ПЛИС на версию, которая может провести синтез структуры с использованием новейших схемотехнических решений и полностью использовать потенциал новых аппаратных платформ. Именно поэтому наиболее важной задачей является обеспечение адекватной поддержки новой продукции, а не получение абсолютно оптимальных решений.

Можно привести конкретные примеры смены требований к средствам синтеза при изменении архитектуры ПЛИС. Например, при переходе к технологическим нормам 90 нм (например, в Spartan-3) стало нецелесообразно выполнять на кристалле внутренние буферы с тремя состояниями. Ранее в ПЛИС Xilinx такие буферы входили в состав каждой логической ячейки и служили для организации сложных коммутационных узлов, например мультиплексоров с большим числом входов. Сравнивая такое решение с мультиплексорами на комбинаторной логике, можно было заметить, что для комбинаторной логики увеличение числа мультиплексируемых входов приводило к вовлечению в процесс мультиплексирования новых логических ячеек. В итоге мультиплексор с большим числом входов реализовался с дополнительными ступенями, объединяющими результаты предварительной обработки, что приводило к увеличению общей задержки распространения сигнала. В то же время буферы с тремя состояниями допускали простое объединение выходов, и, хотя каждый из таких буферов имел несколько большую, чем у комбинаторного мультиплексора, задержку, при нарастании числа входов решение с использованием буферов становилось все более выгодным. Таким образом, средствам синтеза было необходимо определить, какая реализация мультиплексора будет оптимальной в каждом конкретном случае. Однако переход к технологическим нормам 90 нм позволил разместить на кристалле дополнительные узлы, которые существенно облегчили построение таких разветвленных схем, и значимость буферов с тремя состояниями практически исчезла.

Другим примером является эволюция узлов цифровой обработки сигналов. Например, в семействах Virtex-II и Spartan-3 появились аппаратные умножители независимых 18-разрядных операндов. Соответственно, значимость алгоритмов, оптимизирующих реализацию умножения на базе логических ячеек ПЛИС, существенно снизилась, и более важным требованием стала возможность корректной работы средств синтеза с новыми аппаратными узлами, которые по своим возможностям существенно превосходят распределенные умножители на логических ячейках. Аналогично, появление блоков DSP48 с встроенными аккумуляторами уменьшило значимость алгоритмов, оптимизирующих взаимное расположение аппаратного умножителя и аккумулятора на логических ячейках. Завершающим штрихом явилось изменение разрядности операндов с 18×18 на 25×18 в Virtex-5 и расширение возможностей блока DSP48, включая множество режимов, доступных для динамического переключения уже в процессе работы проекта. С точки зрения разработчиков средств синтеза, развитие этих программных продуктов в первую очередь должно включать в себя поддержку новых возможностей аппаратной базы, что является необходимым условием коммерческого успеха, и только во вторую очередь — дальнейшие улучшения уже существующих алгоритмов.

Разумеется, все это не означает, что существующие на рынке средства синтеза не развиваются или не способны дать оптимальное решение. В качестве положительного примера можно привести корректную работу ISE с конструкцией if rising_edge(clk) and ce = '1' then, которая в более ранних версиях приводила к синтезу так называемого gated clock (включение логического вентиля в цепь тактового сигнала, что приводит к целому спектру негативных эффектов). Речь идет исключительно о том, что на пути достижения такого решения перед разработчиками САПР стоит гораздо больше проблем, чем перед разработчиками компиляторов. Соответственно, если ассемблерные вставки в языки высокого уровня, по крайней мере для платформы PC, становятся все более и более редкими, а контраргументом здесь является возможность получения приемлемого решения с помощью оптимизирующего компилятора, то для ПЛИС ситуация является не столь однозначной, и квалифицированный разработчик все еще может существенно улучшить характеристики проекта, не ограничиваясь «включением оптимизации».

В части описания цифровых устройств на HDL можно отметить, что средства синтеза в основном реализуют «то, что написано», не проводя каких-либо принципиальных переработок исходного текста. Иными словами, различные конструкции HDL, даже выполняющие одну и ту же функцию, будут транслироваться в различные цифровые узлы. Рассмотрим основные рекомендации, которых следует придерживаться для получения более эффективных решений в ПЛИС. ■

Продолжение следует